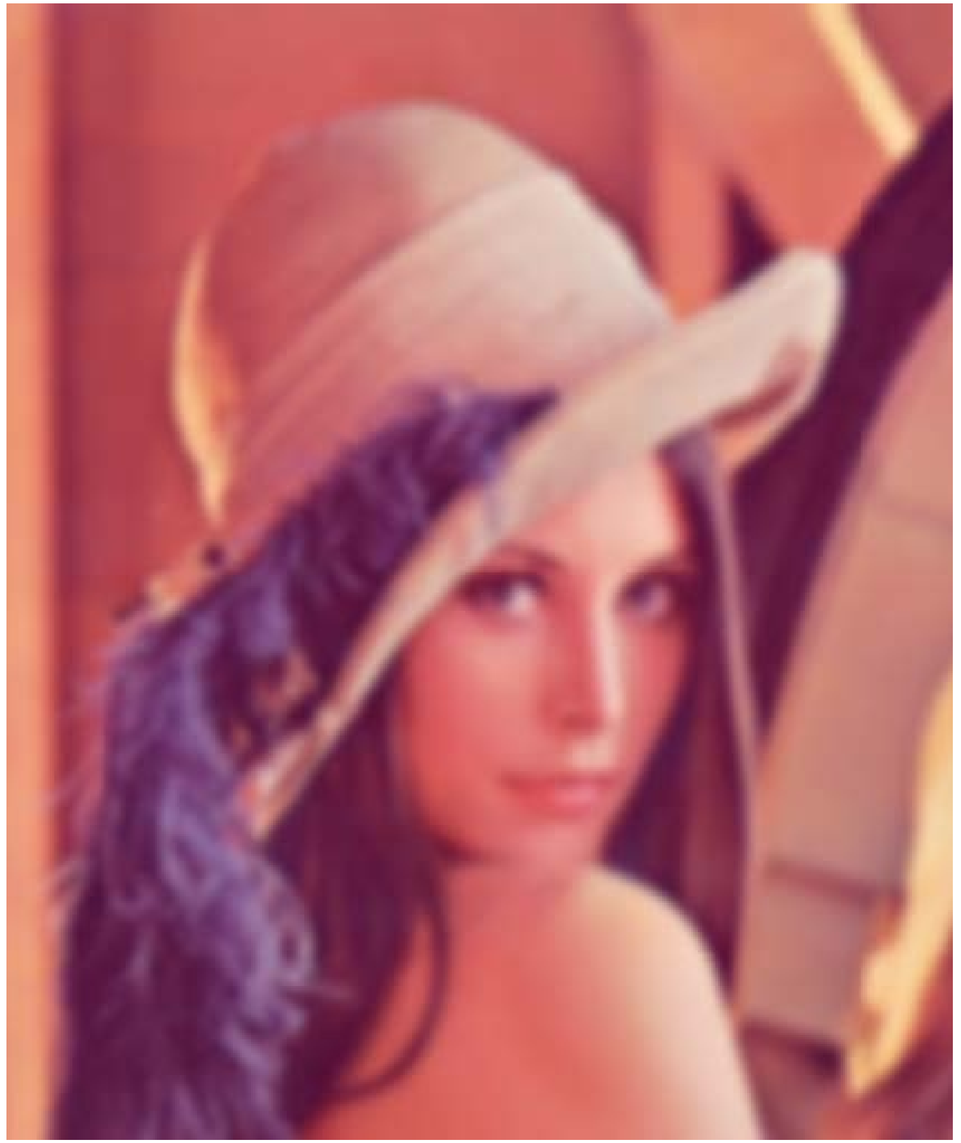
HSLU Lucerne University of Applied Sciences and Arts

Computer Vision & AI

SW 02

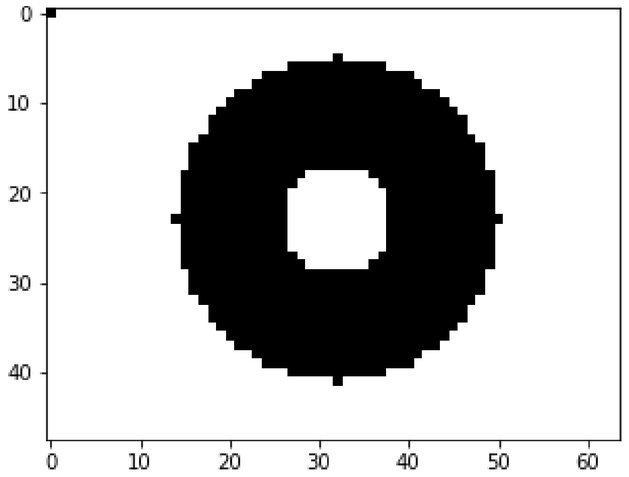
Agenda:

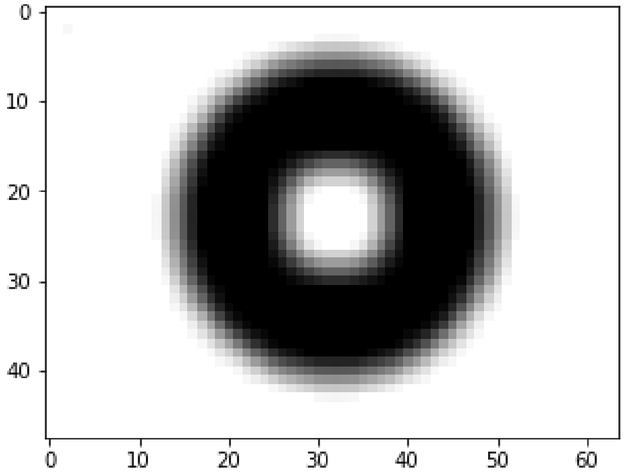
- Smoothing
- Sharpening
- Noise Reduction



26. September 2024

Smoothing





Lowpass Spatial Filters

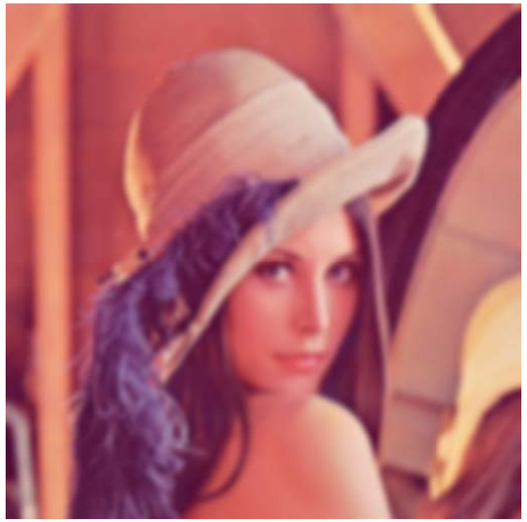
Applications
Antialiasing
Noise Reduction (later)

Neighborhood Operators
compute new image from old image
compute intensity of new pixel from intensities of old pixel and its neighbors

Selected Filters
weighted sum of pixel intensities
kernel: matrix with weights
Box Filter Kernel
Gaussian Kernel

Python Notebook: `1-smoothing.ipynb`

Sharpening

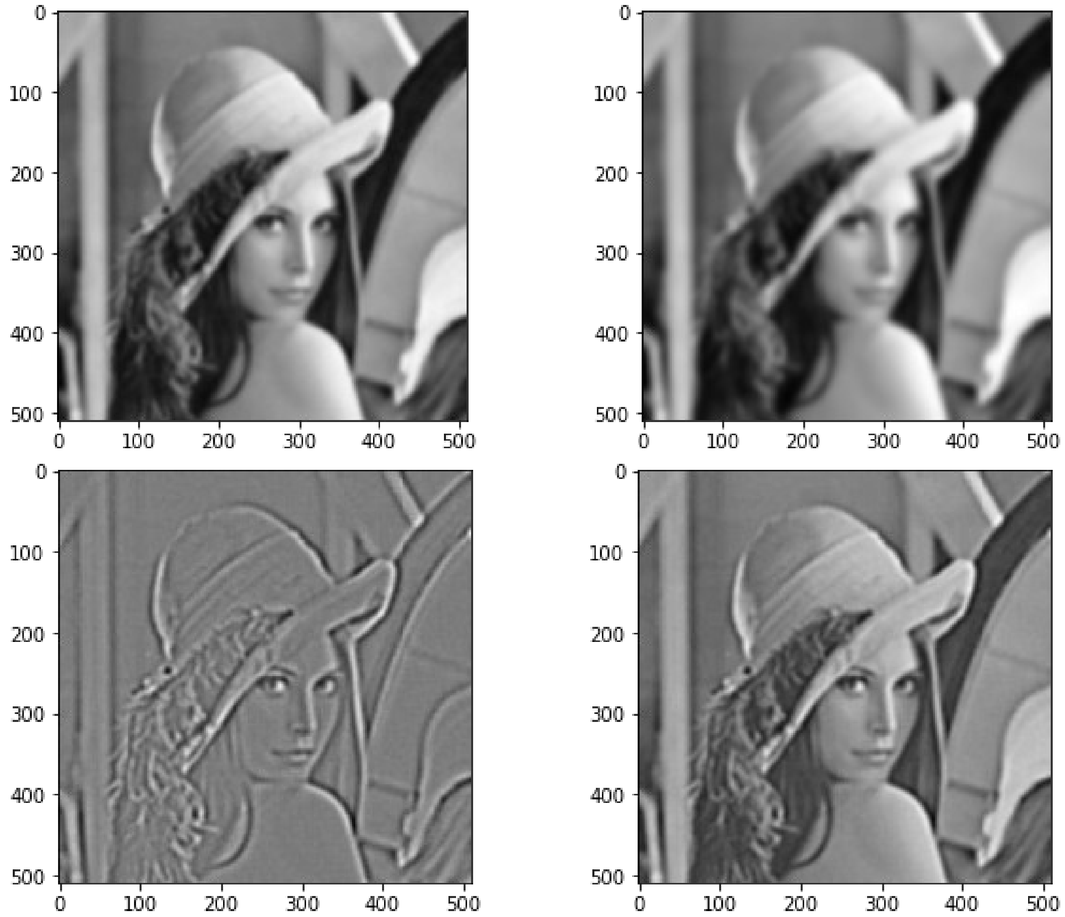


Highpass Spatial Filters

Selected Methods

- Unsharp Masking (with Highboost Filtering)
- Sharpening Using the Heat Equation

Sharpening: Unsharp Masking (with Highboost Filtering)



upper left
Image to Sharpen:

upper right
Image Blurred: (lowpass)

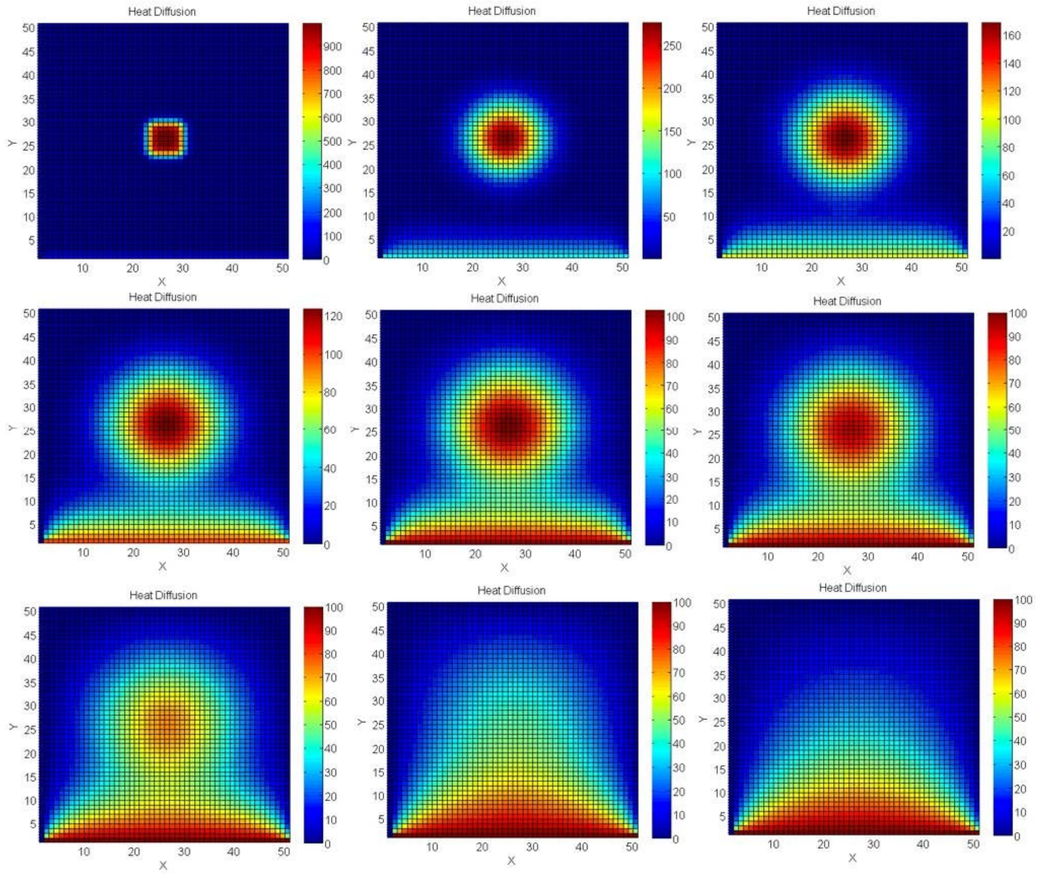
lower left
Mask: (highpass = original - lowpass)

lower right
Sharpened Image:

: Unsharp Masking
: Unsharp Masking with Highboost Filtering

Python Notebook: `2-unsharp-masking.ipynb`

Sharpening: Using the Heat Equation



The 2D Partial Differential Equation

$$\frac{\partial u}{\partial t}(t,x,y) = \alpha \Delta u(t,x,y)$$

$$= \alpha \left(\frac{\partial^2 u}{\partial x^2}(t,x,y) + \frac{\partial^2 u}{\partial y^2}(t,x,y) \right)$$

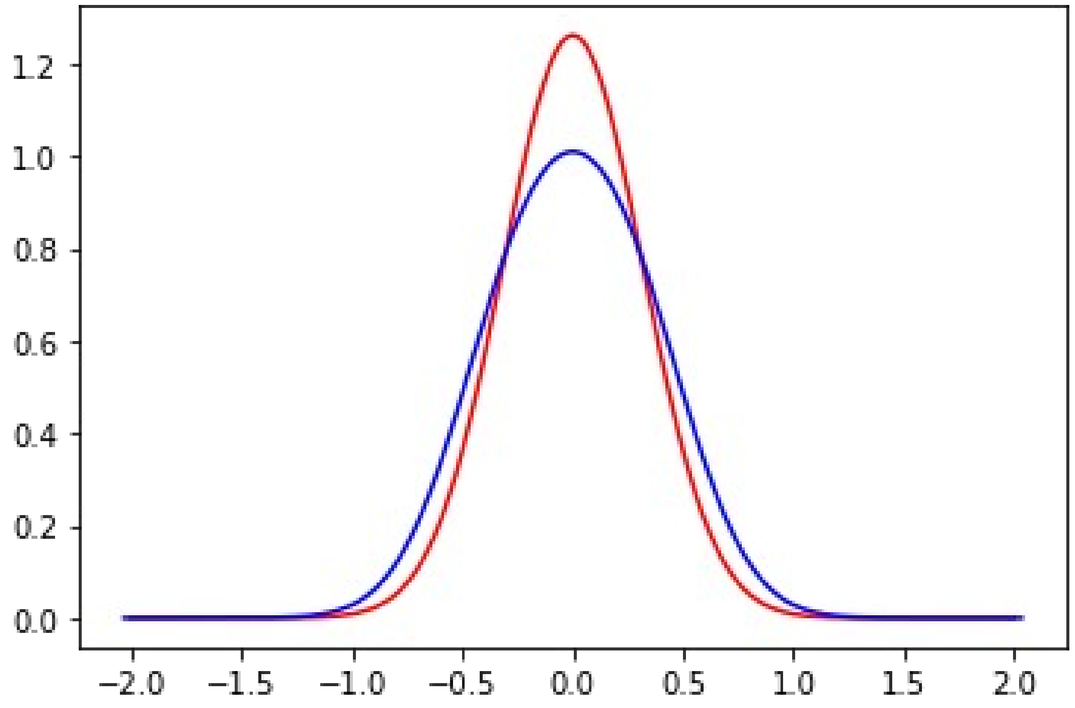
is called

- Heat Equation or
- Diffusion Equation

and describes, how heat diffuses through a given region in 2D.

$\alpha \in \mathbb{R}^+$ is the *thermal diffusivity* of the medium.

Sharpening: 1D Heat Equation



The 1D Heat Equation

$$\frac{\partial u}{\partial t}(t,x) = \alpha \frac{\partial^2 u}{\partial x^2}(t,x)$$

is used to compute the heat diffusion (ordinate)
along a straight line (abscissa) around a hot spot (origin).

The functions (Heat Kernels) $k(a,t,x) = \frac{1}{2\sqrt{\pi a t}} \cdot \mathrm{e}^{-\frac{x^2}{4at}}$

- are solutions of the heat equation.
- show the thermal distribution over time:
  - red function (hot) at initial time
  - blue function (cold) some time later.

Maxima-Skript: `3-heat-equation-1-d.wxmx`

Python Notebook: `4-sharpening-heat-1d.ipynb`

Sharpening: Application of 2D Heat Equation

Diffusion of sharp image $u_s = u(t, x, y)$

leads to blurred image. $u_b = u(t + h, x, y)$

Discretization of Time Derivative:

$$\frac{u(t + h, x, y) - u(t, x, y)}{h} \approx \frac{\partial u}{\partial t}(t, x, y) = \alpha \Delta u(t, x, y)$$

$$\frac{u_b - u_s}{h} \approx \alpha \Delta u(t, x, y)$$

$$u_s \approx u_b - \alpha h \Delta u(t, x, y)$$

$$u_s \approx u_b - c \Delta u(t, x, y) \; ; \qquad c \in \mathbb{R}^+$$

Exercise:

- load blurred Lenna image
- convert it to gray
- compute Laplace operator on blurred image (use discretization on following slides)
- choose proportional constant
- compute sharpened image

Consider using Open CV.

Sharpening: Numerical Differentiation, 1st Derivative

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$ forward difference

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$ backward difference

$$f'(x) \approx \frac{\frac{f(x+h)-f(x)}{h} + \frac{f(x)-f(x-h)}{h}}{2}$$ (arithmetic mean of forward and backward difference)

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$ central difference

Sharpening: Numerical Differentiation, 2nd Derivative and Laplace Operator

$$f''(x) \approx \frac{f'(x+\frac{h}{2}) - f'(x-\frac{h}{2})}{2 \cdot \frac{h}{2}} \qquad \text{(central difference of first derivative, stepsize } \tfrac{h}{2}\text{)}$$

$$f''(x) \approx \frac{\frac{f(x+\frac{h}{2}+\frac{h}{2}) - f(x+\frac{h}{2}-\frac{h}{2})}{2 \cdot \frac{h}{2}} - \frac{f(x-\frac{h}{2}+\frac{h}{2}) - f(x-\frac{h}{2}-\frac{h}{2})}{2 \cdot \frac{h}{2}}}{h} = \frac{f(x+h) - f(x) - f(x) + f(x-h)}{h^2}$$

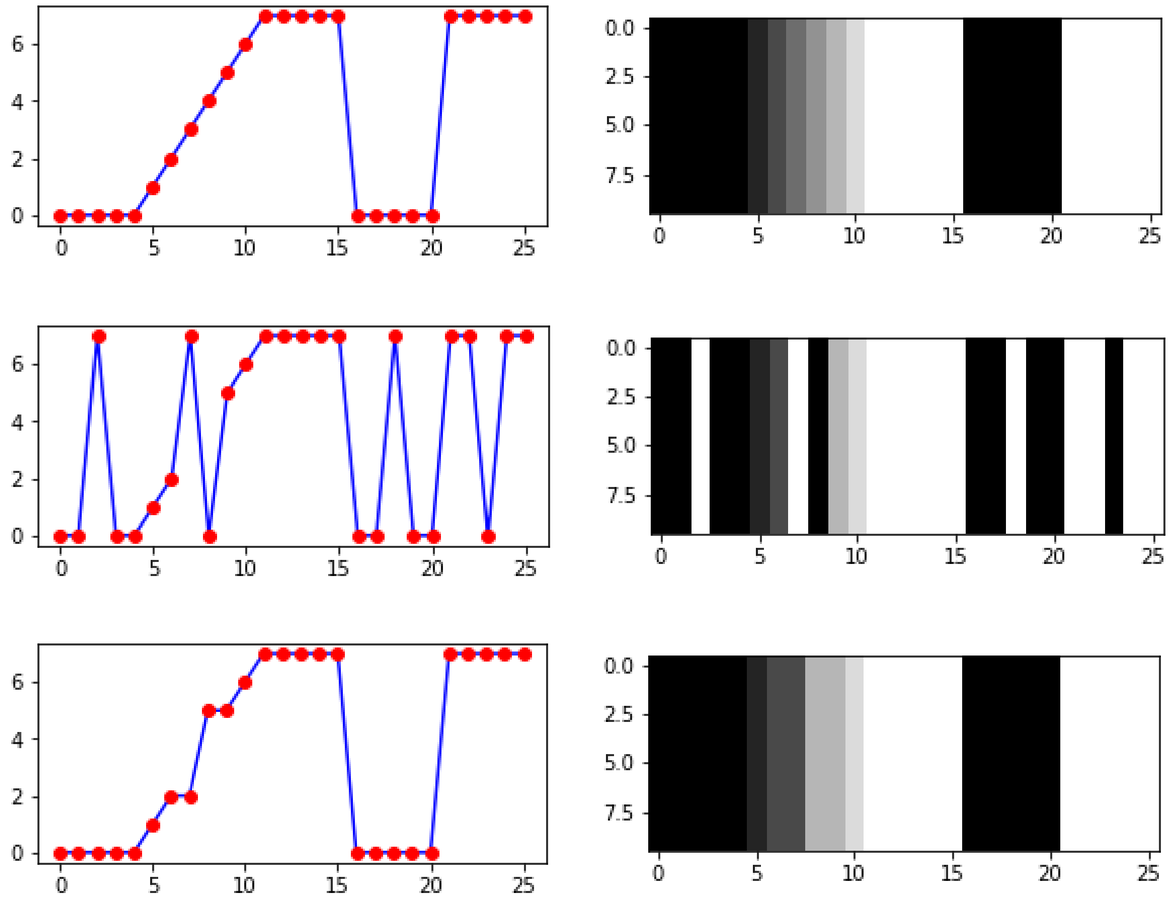
$$f''(x) \approx \frac{f(x+h) - 2\,f(x) + f(x-h)}{h^2}$$

Maxima-Skript: `5-second-derivative-discrete.wxmx`

$$\Delta f(x,y) = \frac{\partial f}{\partial x^2}(x,y) + \frac{\partial f}{\partial y^2}(x,y) \approx \frac{f(x+h,y) - 2\,f(x,y) + f(x-h,y)}{h^2} + \frac{f(x,y+h) - 2\,f(x,y) + f(x,y-h)}{h^2}$$

$$\Delta f(x,y) \approx \frac{1}{h^2}\left(f(x+h,y) + f(x-h,y) + f(x,y+h) + f(x,y-h) - 4\,f(x,y)\right)$$

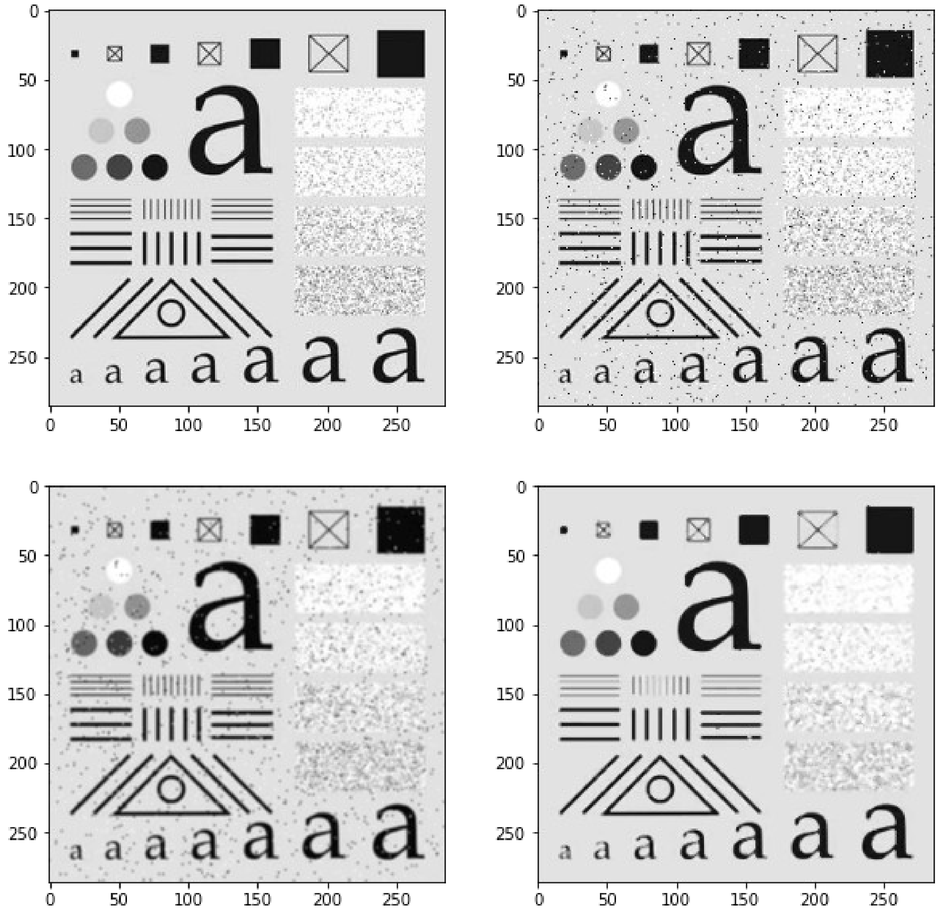
Noise Reduction: 1D Median Filter



- Application: Noise Reduction
- Neighborhood Operator:
  - sort intensities of pixel and its neighbors.
  - take intensity at center of sorted list as result.
- Example:
  - see images left:
    - *upper*: original image
    - *middle*: image with noise
    - *lower*: noisy image after median filtering
  - calculation:
    - pixel 7 has intensity 7
    - left neighbor: pixel 6 with intensity 2
    - right neighbor: pixel 8 with intensity 0
    - intensities [2, 7, 0] sorted: [0, 2, 7]
    - result: 2

Python Notebook: `6-noise-reduction-1d.ipynb`

Noise Reduction: 2D Median Filter vs Gaussian Blur



Exercise:

- load noisy image with test pattern (*upper right)*
- use Open CV to remove noise
  - with Gaussian blur
  - (lower left)
  - with median filter (lower right)
  - change parameters (e. g. neighborhood size)
  - compare results

Summary

- Smoothing
  - Box Kernel: Mean Value
  - Gaussian Kernel: Weighted Sum

  of Pixel's and Neighborhood Intensities
- Sharpening
  - Unsharp Masking (with Highboost Filtering)
  - Sharpening Using the Heat (Diffusion) Equation
- Noise Reduction
  - Gaussian Blur
  - Median